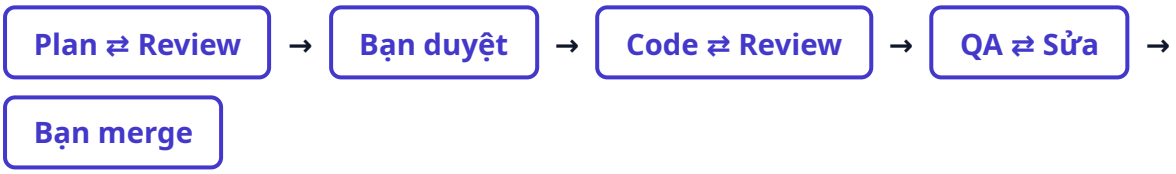
QUANGVY ORCHESTRATION

agent-brain

Hướng dẫn sử dụng — CLI, Dashboard web, Quy trình và Autopilot

Một file binary duy nhất để giao việc cho AI agent theo giao thức Lead → `.plan.md` → Dev → QA, nay có thêm hai vai trò review, chế độ Dev sửa theo báo cáo và autopilot chạy hết vòng lặp nhưng luôn dừng để bạn duyệt plan. Đúng dự án, đúng vai trò, kill sạch cây tiến trình, đo được token và chi phí. Tài liệu này dành cho người dùng hằng ngày và cho việc review, báo lỗi, đề xuất nâng cấp.



Phiên bản phần mềm	**agent-brain 0.1.0** (binary tĩnh musl, Linux x86_64)
Phạm vi	Phase 1–3 (giao thức, lõi CLI), Phase 5 (Dashboard nhúng), Phase 5.1 (Pipeline View) và Phase 5.2 (Quy trình và Autopilot) — **cập nhật đến Phase 5.2**
Ngày	2026-10-21
Mã nguồn	github.com/vinhhx/quang-vy-ai-orchestration

Mục lục

1. Tổng quan: agent-brain làm gì
2. Cài đặt và kiểm tra
3. Chuẩn bị: dự án, profile, cấu hình, quy trình (workflows.json)
4. Làm việc bằng CLI: chạy tay, vai trò review và autopilot
5. Lệnh nhanh, dừng tiến trình và thống kê
6. Dashboard web: khởi động và đăng nhập
7. Dashboard: năm màn hình (pipeline, autopilot, sổ lượt chạy)
8. Dữ liệu, file và an toàn (guard, chế độ sửa)
9. Xử lý sự cố
10. Hạn chế đã biết (trước khi báo lỗi)
11. Review, báo lỗi và đề xuất nâng cấp
12. Phụ lục: mã thoát, khóa cấu hình, HTTP API

Quy ước trong tài liệu

`PROJECT` là tên, id hoặc alias dự án (gõ tiếng Việt có dấu hoặc không dấu đều được). `N` hoặc `TASK_ID` là số task. `~` là thư mục home của bạn. Lệnh in trong khung tối là lệnh gõ ở terminal. Chữ in nghiêng trong dấu nháy là chữ hiện đúng nguyên văn trên màn hình hoặc terminal.

Có gì mới trong bản này (Phase 5.1 và 5.2)

13. **5.1 Pipeline View:** màn hình task có luồng pipeline dạng thẻ, chú thích trạng thái và *Sổ lượt chạy* theo chu kỳ (mục 7.3).
14. **5.2 Quy trình:** mỗi task chạy theo một quy trình có tên (`mac-dinh` , `nhanh` , hoặc quy trình của bạn trong `workflows.json`) (mục 3.4).
15. **5.2 Vai trò mới:** Review plan, Review code, và Dev ở chế độ sửa theo báo cáo (mục 4.2).
16. **5.2 Autopilot:** chạy hết các vòng lặp trong giới hạn 3 vòng / 60 phút / $20, luôn dừng ở bước bạn duyệt plan (mục 4.3–4.5, 7.3).

1. Tổng quan: agent-brain làm gì

agent-brain là bộ điều phối nhẹ, chạy bằng **một file binary**, dùng để giao việc cho AI agent (hiện tại là Claude Code CLI) trên các dự án của bạn. Nó thay thế bộ script Python `brain` cũ và dùng chung danh mục dự án với bộ script đó.

Giao thức và các vai trò

Mỗi task chạy theo một **quy trình** (workflow). Quy trình mặc định `mac-dinh` gồm ba vòng lặp và một cổng duyệt của bạn:

1. Lead	2. Review plan	3. Bạn duyệt plan
Đọc yêu cầu, tạo nhánh `task/<slug>`, viết `.plan.md` và commit `<slug>: plan`. Chỉ được ghi `.plan.md`.	Đọc toàn repo (chỉ đọc), ghi `.plan-review.md`: *approve* hoặc *revise*. Khi *revise*, Lead sửa plan theo báo cáo (vòng 2, 3...).	Autopilot **luôn dừng** ở đây. Bạn bấm *Duyệt plan*, hoặc *Trả về Lead* kèm ghi chú.

4. Dev (Code)	5. Review code	6. QA	7. Sửa
Sửa đúng các file trong bảng *Thay đổi* của plan. Sửa file ngoài danh sách là vi phạm giao thức (mã thoát 3).	Đọc diff từ commit plan, ghi `.code-review.md`: *pass* hoặc *revise*. Khi *revise*, Dev sửa theo báo cáo.	Kiểm tra `git diff`, ghi `.qa-report.md`: *Đạt* hoặc *Cần sửa*. Không sửa code.	Dev ở chế độ sửa theo báo cáo QA, rồi QA chạy lại.

Sau khi QA **Đạt**, bạn tự review và merge nhánh `task/<slug>`. agent-brain không tự merge và không tự tạo PR (*PR nháp · chưa tự động*).

Chạy tay hay autopilot

Cách	Hành vi
Chạy tay	Bạn bấm hoặc gõ lệnh cho từng bước. Quy trình `nhanh` mặc định chạy tay: plan → Bạn → code → QA, không review.
Autopilot	agent-brain tự chạy bước kế tiếp theo quy trình, trong giới hạn số vòng, phút và đô-la. Nó **luôn dừng ở bước bạn duyệt plan**. Khi chạm giới hạn hoặc gặp sự cố, nó dừng với một lý do bằng tiếng Việt và chờ bạn (mục 4.4).

Những đảm bảo chính

- **Đúng dự án (Fallback Ask):** tên dự án khớp 0 hoặc nhiều hơn 1 dự án thì dừng và in danh sách để bạn chọn (mã thoát 2). Không bao giờ đoán.
- **Đúng nhánh:** autopilot và lệnh duyệt có chạy tiếp từ chối chạy khi đang checkout nhánh của task khác (mục 4.5).
- **Kill sạch:** mỗi lần chạy agent nằm trong process group riêng; `kill` dừng cả cây tiến trình con và cả autopilot của task, kiểm tra PID chưa bị tái sử dụng trước khi gửi tín hiệu.
- **Đo được token và chi phí:** mọi sự kiện stream-json được lưu vào SQLite; `stats`, màn hình Thống kê và Sổ lượt chạy tổng hợp token và chi phí tương đương.
- **Hai tài khoản tách biệt:** profile `work` (`~/.claude`) và `personal` (`~/.claude-orca`), gán theo dự án.

Hai cách dùng

Cách	Khi nào dùng
CLI (`agent-brain run …`, `auto`, `approve`)	Làm việc trong terminal, script hóa, chạy trên máy không có trình duyệt.
Dashboard (`agent-brain serve`)	Xem pipeline và log trực tiếp, giao việc và chọn quy trình, duyệt plan, bật/dừng autopilot, bấm Kill, xem thống kê, sửa alias dự án. Chạy ngay trong binary, chỉ trên `127.0.0.1`.

Hai cách dùng chung một cơ sở dữ liệu: run hoặc autopilot khởi chạy ở terminal hiện lên dashboard ngay, và có thể Kill từ dashboard.

Chi phí thực tế ghi nhận khi nghiệm thu Phase 5.2 (repo thử nghiệm nhỏ, Claude thật): một vòng `mac-dinh` đầy đủ có một lần sửa plan (7 run: Lead, Review plan, Lead, Review plan, Dev, Review code, QA) tốn khoảng **$0.88–$1.02** (chi phí tương đương theo API). Task lớn hơn sẽ tốn hơn.

2. Cài đặt và kiểm tra

Yêu cầu

- Linux x86_64. Binary là file tĩnh, không cần thư viện hay runtime nào khác.
- **Claude Code CLI** (`claude`) đã đăng nhập cho profile bạn dùng. Cho profile `personal` : đăng nhập với `CLAUDE_CONFIG_DIR=~/.claude-orca` .
- **git** trong `PATH` .
- Trình duyệt bất kỳ (chỉ cần cho dashboard).

Cài binary

```
# Build từ mã nguồn (offline, khóa phiên bản)
bash scripts/build-release.sh

# Cài vào ~/.local/bin (thư mục này phải có trong PATH)
install -m 755 agent-brain/target/x86_64-unknown-linux-musl/release/agent-brain
~/.local/bin/agent-brain

# Kiểm tra
agent-brain --version       # agent-brain 0.1.0
agent-brain --help
```

Nếu máy không có `musl-gcc` , script build tự chuyển sang `gcc` và in cảnh báo; kết quả vẫn là binary tĩnh.

Lần đầu chạy bản 5.2 trên một `brain.db` cũ, cơ sở dữ liệu được nâng cấp tại chỗ (thêm cột quy trình, duyệt plan, autopilot) mà không mất dữ liệu. Task tạo trước Phase 5.2 vẫn xem được nhưng chạy tay như cũ (không có quy trình, không bật được autopilot).

Danh sách lệnh

Lệnh	Việc làm
`projects scan`	Quét thư mục gốc (độ sâu 1) tìm repo git, cập nhật danh mục. Giữ nguyên phần bạn đã sửa tay.
`projects add`	Thêm thủ công một dự án (`--type code|media`).
`run lead|dev|qa`	Chạy một vai trò của giao thức (Lead lập plan, Dev code hoặc sửa, QA kiểm tra).
`run plan-review`	**Mới (5.2).** Review plan, chỉ ghi `.plan-review.md` .
`run code-review`	**Mới (5.2).** Review code, chỉ ghi `.code-review.md` .
`auto`	**Mới (5.2).** Autopilot cho một task: chạy tiếp, chạy thêm 1 vòng, trả về Lead; hoặc `auto --start` để tạo task mới.
`approve`	**Mới (5.2).** Duyệt plan của task; với quy trình autopilot thì chạy tiếp ngay trong terminal.
`status`	Trạng thái git và run gần nhất. 0 token.
`inspect`	Nhờ Claude đọc nhanh diff chưa commit (1 lượt, không lưu session).
`kill`	Dừng task đang chạy và cả cây tiến trình con; nếu task có autopilot, autopilot cũng dừng.
`stats`	Thống kê token và chi phí tương đương.

`serve`	Mở dashboard web.
`guard`	Hook nội bộ do Claude CLI gọi để chặn ghi file ngoài phạm vi và chặn lệnh Bash không an toàn. Bạn không cần gọi trực tiếp.

3. Chuẩn bị: dự án, profile, cấu hình, quy trình

3.1 Danh mục dự án

File: `~/.config/agent-brain/projects_map.json`. Cách nhanh nhất là quét thư mục chứa các repo:

```
agent-brain projects scan --root ~/code --root ~/work
agent-brain projects add ~/code/kenh-video --type media
```

Mỗi dự án có dạng:

```
{
  "id": "web-ban-hang",
  "path": "/duong/dan/toi/repo",
  "name_vi": "Web bán hàng",
  "aliases": ["shop"],
  "type": "code",
  "repo": null,
  "profile": "work",
  "missing": false,
  "workflow": "nhanh"          ← không bắt buộc (5.2)
}
```

Trường	Ý nghĩa
`id`	Định danh duy nhất (slug).
`name_vi`, `aliases`	Tên và biệt danh để gọi dự án. Có thể gõ có dấu hoặc không dấu. Sửa alias được trên dashboard (màn hình Dự án).
`profile`	**Bắt buộc**: `work` hoặc `personal`. Dự án có `profile: ""` sẽ bị từ chối chạy agent.
`missing`	`true` khi thư mục không còn trên đĩa; dự án này bị bỏ qua khi chọn.
`workflow`	Quy trình mặc định cho task mới của dự án (mục 3.4). Không ghi thì dùng `mac-dinh`. Chỉ sửa trong file; dashboard không sửa trường này.

3.2 Profile tài khoản

Profile	Thư mục cấu hình Claude	Ghi chú
`work`	`~/.claude` (mặc định)	Tài khoản công việc.
`personal`	`~/.claude-orca`	**Tối đa 1 run cùng lúc** trên toàn hệ thống (quy tắc E-06), vì thư mục này có thể dùng chung với phiên Claude bạn đang mở. Áp dụng cả cho autopilot.

Khi chạy agent, agent-brain bỏ đặt `ANTHROPIC_API_KEY`, `ANTHROPIC_AUTH_TOKEN`, `ANTHROPIC_BASE_URL` để agent luôn dùng đúng tài khoản đã đăng nhập của profile.

> **Không ghi đè profile khi giao việc**
> Profile luôn lấy từ dự án. Muốn đổi tài khoản cho một dự án, sửa trường `profile` trong `projects_map.json` (dashboard không cho sửa trường này).

3.3 File cấu hình (tùy chọn)

File `~/.config/agent-brain/config.json`. Không có file thì dùng mặc định nhúng sẵn. Các khóa hay chỉnh (danh sách đầy đủ ở Phụ lục B):

Khóa	Mặc định	Ý nghĩa
`models`	lead: opus, dev/qa: sonnet; plan-review / code-review: opus (khi không ghi)	Model theo vai trò. Ví dụ đổi review sang sonnet: `"plan-review": "sonnet"`.
`timeout_seconds`	lead/dev 1800, qa 1200, plan-review 1200, code-review 1500, inspect 300	Thời gian chạy tối đa của một run (giây); quá hạn thì run kết thúc `timed_out`. Vẫn là trần cứng khi autopilot chạy.
`autopilot`	`max_rounds` 3, `budget_minutes` 60, `budget_usd` 20	Giới hạn mặc định cho mọi quy trình (mục 3.5).
`pricing`	bảng giá opus / sonnet / haiku	Dùng để ước tính chi phí của run không có kết quả cuối khi tính ngân sách (mục 3.5).
`kill_grace_secs`	3	Chờ giữa SIGINT và SIGKILL khi dừng.
`dashboard_port`	7437	Cổng dashboard.
`claude_bin`	`claude`	Đường dẫn Claude CLI.

Một `config.json` viết trước Phase 5.2 (không có khóa của hai vai trò review) vẫn dùng được: hai vai trò review nhận timeout 1200 và 1500 giây, model opus. Biến môi trường: `AGENT_BRAIN_CONFIG` (file cấu hình khác), `AGENT_BRAIN_DB` (database khác, tiện để thử nghiệm cách ly).

3.4 Quy trình (workflows.json)

Mỗi task chạy theo một quy trình có tên. Có sẵn hai quy trình nhúng trong binary:

Tên	Giai đoạn	Tóm tắt (như dashboard hiện)	Chạy
`mac-dinh`	`plan`, `plan-review`, `gate:you`, `implement`, `code-review`, `qa`, `fix` — ba vòng lặp	plan ⇄ review → Bạn → code ⇄ review → QA ⇄ sửa · 3 vòng · 60 phút · $20	**autopilot** (`auto: true`)
`nhanh`	`plan`, `gate:you`, `implement`, `qa`	plan → Bạn → code → QA · 60 phút · $20	**chạy tay** (`auto: false`)

Thêm hoặc ghi đè quy trình trong `~/.config/agent-brain/workflows.json` (khi đặt `XDG_CONFIG_HOME` thì là `$XDG_CONFIG_HOME/agent-brain/workflows.json`; `AGENT_BRAIN_CONFIG` không đổi vị trí file này). File không bắt buộc, tối đa 256 KiB, được đọc lại mỗi lần cần (không phải khởi động lại `serve`). Dashboard chỉ xem; sửa bằng editor.

Khóa	Ý nghĩa
`stages` (bắt buộc)	Danh sách giai đoạn, chỉ gồm `plan`, `plan-review`, `gate:you`, `implement`, `code-review`, `qa`, `fix`; không lặp, đúng thứ tự đó.
`loops`	Chỉ nhận ba cặp `"plan": "plan-review"`, `"implement": "code-review"`, `"qa": "fix"`. Không ghi thì mỗi cặp có đủ hai giai đoạn tự thành vòng lặp.
`max_rounds`	Số vòng tối đa mỗi vòng lặp, số nguyên 1–10.

`budget_minutes`	Tổng phút của task, số nguyên 1–1440.
`budget_usd`	Tổng chi phí của task, lớn hơn 0, tối đa 1000.
`auto`	`true` : Giao việc trên dashboard tự bật autopilot cho task mới. Mặc định `false` .

Ba khóa giới hạn không ghi thì lấy từ khóa `autopilot` của `config.json` . Khóa nào ngoài danh sách là lỗi. Tên quy trình: chữ thường, số và dấu gạch ngang, tối đa 40 ký tự.

Luật cổng duyệt: quy trình có `plan` hoặc `plan-review` mà còn bước sau (code, review code, QA, sửa) thì **bắt buộc** có `gate:you` . Không quy trình nào bỏ qua được bước bạn duyệt plan. `gate:you` cần có `plan` ; `fix` cần có `qa` .

Ví dụ hợp lệ — thêm quy trình `chi-phi-thap` và rút ngắn `nhanh` (ghi đè theo tên phải ghi lại đủ `stages`):

```
{
  "chi-phi-thap": {
    "stages": ["plan", "plan-review", "gate:you", "implement", "qa"],
    "max_rounds": 2,
    "budget_usd": 3,
    "auto": true
  },
  "nhanh": {
    "stages": ["plan", "gate:you", "implement", "qa"],
    "budget_minutes": 30,
    "auto": false
  }
}
```

Kết quả: `chi-phi-thap` = *plan ⇄ review → Bạn → code → QA · 2 vòng · 60 phút · $3*; `nhanh` thành *30 phút · $20*. Màn hình Dự án gắn nhãn *từ workflows.json* và *ghi đè quy trình có sẵn*.

Ví dụ không hợp lệ — bỏ bước duyệt và số vòng ngoài khoảng:

```
{
  "bo-qua-duyet": {
    "stages": ["plan", "implement", "qa"],
    "max_rounds": 0
  }
}
```

agent-brain bỏ riêng quy trình này và báo bằng tiếng Việt kèm số dòng:

```
Dòng 3: quy trình "bo-qua-duyet" phải có "gate:you" giữa bước plan và bước code/QA —
autopilot luôn dừng ở bước bạn duyệt plan.
Dòng 4: "max_rounds" phải là số nguyên từ 1 đến 10.
```

- File sai cú pháp JSON bị bỏ cả file: *"Dòng L, cột C: workflows.json không phải JSON hợp lệ."*
- Một bản ghi đè sai thì quy trình có sẵn giữ nguyên. Hai quy trình `mac-dinh` và `nhanh` **luôn dùng được**.
- `run lead` in mỗi lỗi thành dòng *"Cảnh báo workflows.json: …"*; Giao việc và Dự án hiện khung màu hổ phách *"workflows.json có lỗi"*.

Chọn quy trình cho một task

Thứ tự ưu tiên: **cờ `--workflow`** (CLI) hoặc lựa chọn ở ô *Quy trình* trên Giao việc → **`workflow` của dự án** trong `projects_map.json` → `mac-dinh`.

- Tên truyền qua `--workflow` mà không tồn tại: lệnh báo lỗi *"Không có quy trình 'x' — các quy trình có: …"*, không tạo nhánh hay task.
- Tên ghi ở dự án mà không tồn tại: dùng `mac-dinh` và báo *"Quy trình 'x' của dự án không có trong workflows.json — dùng mac-dinh."*
- **Bản sao theo task:** khi tạo task, quy trình đã chọn (kèm giới hạn) được lưu cùng task. Sửa `workflows.json` sau đó **không** ảnh hưởng task đang có — kể cả khi bạn tăng ngân sách.

3.5 Giới hạn, ngân sách và bảng giá

Giới hạn	Mặc định	Cách tính
Số vòng (`max_rounds`)	3	Cho mỗi vòng lặp. Một vòng = một lần chạy của tác giả (Lead / Dev) cùng lần kiểm tra theo sau. Một bước review hoặc QA không có bước tác giả / Sửa đi kèm chỉ chạy 1 lần.
Phút (`budget_minutes`)	60	Tổng thời lượng mọi run của task (run đang chạy tính đến hiện tại; `inspect` không tính).
Đô-la (`budget_usd`)	20	Tổng chi phí tương đương của task. Run có kết quả cuối dùng chi phí Claude CLI báo; run bị Kill, lỗi hoặc chỉ có token một phần được ước tính từ token theo bảng `pricing`.

Ngân sách được kiểm tra **trước khi bắt đầu mỗi bước**: run đang chạy luôn được chạy xong, rồi autopilot không bắt đầu bước mới. Dashboard ghi *sắp hết* từ 80 % và *đã hết* từ 100 % giới hạn.

Bảng giá mặc định (USD cho mỗi triệu token, khóa `pricing` trong `config.json`):

Họ model	input	output	cache_read	cache_write
`opus`	4.0	20.0	0.2	8.0
`sonnet`	2.0	10.0	0.1	4.0
`haiku`	0.1	0.5	0.005	0.2

Ước tính = (input × giá input + output × giá output + cache_read × giá cache_read + cache_write × giá cache_write) / 1.000.000. Ghi đè từng họ trong `config.json` (phải đủ 4 số không âm); họ không ghi giữ giá mặc định. Model không khớp họ nào được tính theo họ đắt nhất để không vượt trần. Đây là bảng nhúng sẵn — hãy sửa nếu khác bảng giá bạn dùng.

4. Làm việc bằng CLI: chạy tay, vai trò review và autopilot

4.1 Chạy tay từng bước (Lead → Dev → QA)

Bước 1 — Lead viết plan

```
agent-brain run lead "web bán hàng" "Thêm nút xuất đơn hàng ra CSV ở trang quản trị"
# tùy chọn: --name xuat-csv      (đặt slug cho task)
#           --workflow nhanh     (chọn quy trình cho task mới, mục 3.4)
#           --dry-run            (chạy thử, không sinh tác vụ)
#           --verbose            (in chi tiết)
```

Kết quả: nhánh `task/<slug>`, file `.plan.md` ở gốc repo và commit `<slug>: plan`. Terminal in cặp **task N, run M** — ghi lại số task để dùng khi duyệt, chạy autopilot, kill hoặc báo lỗi. Tên dự án mơ hồ thì lệnh dừng với mã 2 và in danh sách dự án khớp. `run lead` chỉ chạy Lead; nó không bật autopilot (dùng `auto`, mục 4.3).

Bước 2 — Bạn duyệt plan

- Mở `.plan.md` trên nhánh task. Kiểm tra bảng **Thay đổi**: Dev chỉ được sửa các file liệt kê ở đây.
- Cần sửa plan: sửa bằng editor rồi **commit**. Plan có thay đổi chưa commit sẽ chặn bước Dev và lệnh duyệt.
- Ghi nhận duyệt: `agent-brain approve N` (mục 4.3). Với quy trình chạy tay, trên dashboard bấm *Chạy Dev* trực tiếp cũng được coi là đã duyệt.
- Một commit plan mới sau khi duyệt (kể cả sửa tay) đưa cổng về *chờ bạn*: plan chưa được bạn xem thì không chạy tiếp.

Bước 3 — Dev thực hiện

```
agent-brain run dev "web bán hàng"
```

Dev làm việc trên nhánh task hiện tại và chỉ được ghi các file trong plan. Có hai kết thúc bất thường:

- **Vướng mắc** (mã thoát 4): Dev thiếu thông tin, ghi mục *Vướng mắc* vào `.plan.md` và dừng an toàn. Bạn bổ sung plan, commit, rồi chạy lại Dev (hoặc chạy lại Lead).
- **Vi phạm giao thức** (mã thoát 3): có file ngoài phạm vi bị sửa. Xem lại diff trước khi làm tiếp.

Bước 4 — QA kiểm tra

```
agent-brain run qa "web bán hàng"
```

QA đọc `git diff` từ commit plan đến HEAD (bỏ qua các file giao thức `.plan.md`, `.qa-report.md`, `.plan-review.md`, `.code-review.md`) và ghi `.qa-report.md` với kết luận **Đạt** hoặc **Cần sửa**, kèm số liệu `specs` (số kiểm thử) và `failing` (số lỗi) ở đầu file. **Cần sửa** → chạy Dev ở chế độ sửa (`--fix qa`, mục 4.2) hoặc chạy lại Lead.

> **Mẹo**
> Trước khi chạy QA, dùng `agent-brain status PROJECT` (0 token) để xem nhánh và diffstat, hoặc `agent-brain inspect PROJECT` để nhờ Claude đọc nhanh diff chưa commit.

4.2 Vai trò review và chế độ sửa

Lệnh	Việc làm
`agent-brain run plan-review PROJECT`	Review plan (mặc định opus). Đọc được toàn repo, **chỉ ghi `.plan-review.md`**. Kết luận `approve` hoặc `revise`, kèm số mục *chặn* (`blocking`) và *góp ý* (`advisory`). `revise` chỉ hợp lệ khi có ít nhất một mục chặn. Báo cáo được commit.
`agent-brain run code-review PROJECT`	Review code (mặc định opus). Đọc diff code từ commit plan, **chỉ ghi `.code-review.md`**: `pass` hoặc `revise`, kèm số mục chặn / góp ý. Từ chối chạy khi chưa có thay đổi code.
`agent-brain run lead PROJECT "…" --name SLUG --review`	Lead sửa trực tiếp `.plan.md` đã commit và phải xử lý mọi mục chặn trong `.plan-review.md` (cần báo cáo có kết luận `revise`).
`agent-brain run lead PROJECT "…" --name SLUG --note="…"`	Gửi ghi chú của bạn cho Lead khi lập lại plan (tối đa 500 ký tự, lưu cùng run). Dùng đúng dạng `--note=` để ghi chú bắt đầu bằng `-` vẫn đúng. Lập lại plan xóa bản duyệt cũ.
`agent-brain run dev PROJECT --fix code-review`	Dev ở chế độ sửa, đọc `.code-review.md` làm đầu vào. Sau đó chạy lại Review code.
`agent-brain run dev PROJECT --fix qa`	Dev ở chế độ sửa, đọc `.qa-report.md` làm đầu vào. Sau đó chạy lại QA.

Terminal in kết luận, ví dụ *"Kết luận Review plan: revise (1 chặn, 1 góp ý)"*, *"Kết luận Review code: pass (0 chặn, 1 góp ý)"*, *"Kết luận QA: Đạt"*. Ở chế độ sửa, terminal in *"File được nêu trong <báo cáo>: …"* và một dòng *"Bỏ qua (không được phép): <đường dẫn> (<mã>)"* cho mỗi file bị từ chối. Phạm vi ghi của chế độ sửa ở mục 8.

4.3 Autopilot bằng CLI

```
# Tạo task mới và chạy autopilot luôn (dừng ở bước duyệt plan)
agent-brain auto --start --workflow mac-dinh --name xuat-csv -- "web bán hàng" "Thêm nút xuất CSV"

# Duyệt plan rồi chạy tiếp Code → Review code → QA → Sửa ngay trong terminal này
agent-brain approve 12

# Chạy tiếp task 12 từ chỗ dừng (ví dụ sau khi run lead tay, sau Ctrl+C, hoặc để bật lại)
agent-brain auto 12

# Sau khi autopilot dừng: cho vòng vừa dừng thêm 1 vòng rồi chạy tiếp
agent-brain auto 12 --more-round

# Trả về Lead (lập lại plan, xóa bản duyệt), kèm ghi chú
agent-brain auto 12 --replan --note="Tách bước 2 thành hai commit nhỏ"

# Chỉ ghi duyệt, không chạy tiếp (ở nhánh nào cũng được)
agent-brain approve 12 --no-continue
```

Cờ	Ghi chú
`auto --start [--name SLUG] [--workflow NAME] -- PROJECT REQUEST`	Đặt dự án và yêu cầu sau `--`. `--name`, `--workflow` chỉ dùng với `--start`.

`auto N --more-round`	Chỉ khi autopilot đã dừng. Với *vi phạm giao thức*, *dừng bởi bạn*, *run lỗi*: chạy lại bước đó. Sau *hết ngân sách*: cho chạy vượt ngân sách đúng một bước tác giả và một bước kiểm tra, rồi giới hạn áp dụng lại.
`auto N --replan [--note=TEXT]`	Trả về Lead. `--note` chỉ dùng kèm `--replan`. Sau đó autopilot chạy lại vòng plan ⇄ review plan và lại dừng ở cổng.
`approve N [--no-continue]`	Ghi duyệt plan hiện tại (gắn với đúng commit plan). Từ chối khi plan chưa commit, `.plan.md` có sửa chưa commit, đang có run hoặc autopilot chạy, hoặc task tạo trước Phase 5.2.

Mỗi bước autopilot in một dòng *"Autopilot · task N · plan-review vòng 1"*. Các dòng kết thúc:

Terminal in	Mã thoát	Nghĩa
Autopilot dừng ở bước duyệt plan (task N) — duyệt: agent-brain approve N	0	Chờ bạn duyệt plan.
Autopilot xong: QA đạt (task N)	0	Xong; bạn review và merge.
Autopilot dừng · {lý do}: {chi tiết} (task N)	4	Dừng và chờ bạn (mục 4.4).
Autopilot đã dừng (bị ngắt) — task N; chạy tiếp: agent-brain auto N	130	Bị Ctrl+C / SIGTERM / SIGHUP.
Autopilot đã tắt cho task N — chỉ ghi nhận duyệt. Bật lại: agent-brain auto N	0	Bạn duyệt khi autopilot đã tắt (xem dưới).

> **Duyệt plan chỉ chạy tiếp khi autopilot còn bật**
> Với quy trình `auto: true, approve` (hoặc nút *Duyệt plan*) chạy tiếp autopilot. Nếu bạn đã tắt autopilot cho task (*Dừng autopilot* lúc đang chờ ở cổng, hoặc *Dừng hẳn* sau một lần dừng), lệnh duyệt **chỉ ghi nhận** duyệt, không chạy gì. Bật lại bằng `agent-brain auto N` hoặc nút *Bật autopilot*; autopilot lại dừng ở cổng nếu plan chưa được duyệt.

Ctrl+C trong terminal của `auto` dừng êm: driver chuyển đúng một SIGTERM cho bước đang chạy (không dừng cưỡng bức), bước đó dừng Claude và ghi kết quả run, rồi driver thoát 130. Chạy lại bằng `agent-brain auto N`. Autopilot mà driver chết bất thường cũng được nhận lại bằng lệnh này.

4.4 Lý do autopilot dừng và việc bạn nên làm

Khi dừng, task vào nhóm *Chờ bạn ở* Tổng quan, màn hình task hiện biểu ngữ màu hổ phách *"Autopilot dừng · {lý do}"* với ba nút **Chạy thêm 1 vòng**, **Chạy lại Lead**, **Dừng hẳn**.

Lý do	Chi tiết hiện ra (nguyên văn mẫu)	Nên làm
hết vòng	*Review plan vẫn yêu cầu sửa lại sau N / cap vòng. · Review code vẫn yêu cầu sửa lại sau N / cap vòng. · QA vẫn kết luận Cần sửa sau N / cap vòng. Xem báo cáo QA bên dưới.*	Đọc báo cáo review / QA cuối. Nếu thấy đang tiến gần: **Chạy thêm 1 vòng**. Nếu plan sai hướng: **Chạy lại Lead**. Muốn tự sửa: **Dừng hẳn**. Nếu dừng ở vòng plan, bạn vẫn có thể duyệt plan hiện tại.
hết ngân sách	*Đã dùng M / B phút · $X / $Y. Run đang chạy đã xong; autopilot không bắt đầu bước nào nữa.*	**Chạy thêm 1 vòng** = vượt ngân sách đúng một lần (một bước tác giả + một bước kiểm tra). Sửa `workflows.json` không đổi ngân sách của task đang có. Hoặc **Dừng hẳn** và làm tay.
vi phạm giao thức	*Run {id} ({vai trò}) vi phạm giao thức — xem log của run này.*	Xem log và `git diff`; hoàn tác thay đổi ngoài phạm vi. Rồi **Chạy thêm 1 vòng** (chạy lại bước đó) hoặc **Chạy lại Lead**.

vướng mắc	*Dev dừng vì vướng mắc. Sửa .plan.md trong editor của bạn rồi commit với message '{slug}: plan', hoặc chạy lại Lead.*	Làm đúng hướng dẫn. Commit plan mới đưa cổng về *chờ bạn*: duyệt lại rồi chạy tiếp. Nút *Chạy thêm 1 vòng* bị khóa khi vướng mắc chưa xử lý.
dừng bởi bạn	*Bạn đã dừng autopilot lúc HH:MM.*	Bạn đã bấm Kill hoặc `agent-brain kill N`. Muốn tiếp: **Chạy thêm 1 vòng** (chạy lại bước bị dừng).
run lỗi	*Run {id} ({vai trò}) lỗi hoặc quá thời gian — xem log của run này.* Hoặc dòng lỗi thật của bước, ví dụ *Không ở nhánh task '...' (nhánh hiện tại: '...') — không đoán*, *Working tree có thay đổi chưa commit: ...*, *Lead đã chạy n vòng mà chưa có commit plan*.	Đọc chi tiết. Sai nhánh: `git checkout task/<slug>` trong repo. Working tree bẩn (thường là `.qa-report.md` không có kết luận): xóa hoặc commit file đó. Rồi **Chạy thêm 1 vòng**.

Ngoài ra driver tự dừng với *run lỗi* khi chạy quá 50 bước trong một lần, hoặc khi tiến trình của một bước thoát với mã lỗi sau khi run đã kết thúc (ví dụ commit bị hook từ chối) — chi tiết là dòng stderr cuối cùng của bước.

4.5 Quy tắc chạy song song và quy tắc nhánh

- Mỗi dự án chỉ có **một run** tại một thời điểm. Profile `personal` chỉ có **một run** trên toàn hệ thống. Các dự án `work` khác nhau có thể chạy song song.
- Mỗi task chỉ có **một autopilot**. Autopilot không bắt đầu khi dự án đang có run hoặc autopilot khác, hay khi profile `personal` đang bận.
- Khi autopilot của một dự án đang sống (kể cả lúc nghỉ giữa hai bước), `run` tay trên dự án đó bị từ chối: *"Dự án ... đang chạy autopilot cho task N — ..."*; với `personal`: *"Profile personal đang chạy autopilot cho task N — ..."*. Trên dashboard, Giao việc và nút chạy tay bị khóa với cùng lý do.
- **Luật nhánh:** mọi bước autopilot và mọi lần duyệt có chạy tiếp cần đang checkout **nhánh của task**. Khác nhánh thì từ chối: *"Không ở nhánh task 'task/<slug>' (nhánh hiện tại: '...') — không đoán"*. `approve --no-continue` và lần duyệt chỉ ghi nhận thì ở nhánh nào cũng được. Bước Dev, Review và QA do autopilot bật cũng tự từ chối chạy trên nhánh của task khác: *"Nhánh hiện tại '...' không phải nhánh của task autopilot N — dừng, không chạy"*.

5. Lệnh nhanh, dừng tiến trình và thống kê

status — 0 token

```
agent-brain status "web bán hàng"
```

Không gọi Claude. Hiển thị trạng thái git, nhánh hiện tại, diffstat và run gần nhất trong database.

inspect — đọc nhanh diff

```
agent-brain inspect "web bán hàng" [--verbose]
```

Gọi Claude đúng 1 lượt, không công cụ, không lưu session. Diff lớn hơn `inspect_max_diff_bytes` (100.000 byte) được cắt gọn và có thông báo.

kill — dừng task (và autopilot)

```
agent-brain kill 12        # 12 là TASK_ID
```

1. Nếu task có autopilot đang chạy, ghi yêu cầu dừng trước, để autopilot dừng với lý do *dừng bởi bạn* thay vì chạy bước kế tiếp.
2. Đọc PID và thời điểm khởi tạo của run từ database, đối chiếu với `/proc`. Nếu PID đã bị hệ điều hành tái sử dụng, **không** gửi tín hiệu.
3. Gửi SIGINT tới cả process group, chờ `kill_grace_secs`, rồi SIGKILL những tiến trình còn sót.
4. Run được ghi `killed`; terminal đang chạy run đó thoát với mã 130, terminal đang chạy `auto` in *"Autopilot dừng · dừng bởi bạn: …"* và thoát 4.

Nếu autopilot đang ở giữa hai bước (không có run nào chạy), lệnh trả 0 với dòng *"Đã yêu cầu autopilot dừng (task N) …"*.

Ở terminal đang chạy `run`: **Ctrl+C một lần** = dừng mềm; **Ctrl+C hai lần nhanh** = dừng ngay (dưới 1 giây). Đóng cửa sổ terminal cũng dừng cả cây tiến trình. Ở terminal `auto`: Ctrl+C dừng êm (mục 4.3).

Trạng thái kết thúc bất thường của run: `killed` (bị dừng có chủ đích), `timed_out` (quá thời gian), `lost` (tiến trình biến mất bất thường).

Báo cáo của Review plan, Review code và QA được tạo khung trước khi Claude chạy. Khi run đó lỗi, quá thời gian hoặc bị Kill, báo cáo được trả về như cũ và terminal in *"Đã dọn <file> của lượt chạy không thành công."*, để bước sau không bị chặn vì working tree bẩn. Báo cáo sai định dạng, báo cáo QA không có kết luận và run vi phạm giao thức thì **giữ nguyên** báo cáo làm bằng chứng (xem mục 10).

stats — token và chi phí

```
agent-brain stats
agent-brain stats --since 7d --profile work
agent-brain stats --since 24h --json      # JSON: total, by_project, by_role
```

`--since` nhận `Nm`, `Nh`, `Nd` (phút, giờ, ngày). Chi phí là **giá trị tương đương theo API** do Claude CLI báo trong kết quả của mỗi run — không phải hóa đơn thật. Run bị Kill hoặc chưa có kết quả không có số chi phí này (ngân sách autopilot ước tính riêng cho chúng, mục 3.5).

6. Dashboard web: khởi động và đăng nhập

Khởi động

```
agent-brain serve              # cổng 7437
agent-brain serve --port 8123  # cổng khác; --port 0 = hệ điều hành tự chọn
```

Terminal in một URL có token, dạng:

```
agent-brain dashboard: http://127.0.0.1:7437/?token=<64 ký tự hex>
```

1. Mở **đúng URL đó** trong trình duyệt (dùng `127.0.0.1`, không dùng `localhost`).
2. Server đổi token thành cookie phiên và chuyển bạn về `/`; thanh địa chỉ không còn token.
3. Giữ terminal mở. **Ctrl+C** dừng dashboard — các run **và các autopilot** do dashboard khởi chạy cũng dừng theo. Autopilot bật bằng `agent-brain auto` trong terminal khác thì vẫn chạy.

Terminal của `serve` cũng in các dòng *Autopilot …* của autopilot do dashboard bật.

Đặc điểm cần biết

- **Token mới mỗi lần khởi động**, không ghi xuống đĩa. Khởi động lại `serve` thì cookie cũ hết hiệu lực — mở URL mới.
- **Chỉ một `serve`** tại một thời điểm. Chạy cái thứ hai sẽ bị từ chối kèm PID và cổng của cái đang chạy.
- **Chỉ nghe `127.0.0.1`**: máy khác trong mạng LAN không vào được. Đây là chủ ý vì dashboard chạy được lệnh trên máy bạn.
- **Chạy offline hoàn toàn**: CSS, JavaScript, font đều nằm trong binary.

> **Không chia sẻ URL có token**
> Ai có token đều điều khiển được agent trên các dự án của bạn. Không dán URL vào chat, issue, log hay ảnh chụp màn hình.

Bố cục chung

- **Thanh trên:** các mục điều hướng (Tổng quan, Giao việc, Thống kê, Dự án; Chi tiết task mở từ thẻ task), bộ lọc profile (**Tất cả / work / personal**) và nút đổi giao diện sáng/tối.
- **Bộ lọc profile chỉ lọc những gì bạn thấy** (task, thống kê, gợi ý dự án). Nó không đổi tài khoản chạy task.
- Dữ liệu tự cập nhật (màn hình task mỗi 3 giây, Tổng quan mỗi 4 giây; log là trực tiếp). Mất kết nối thì hiện *"Không cập nhật được — đang thử lại…"* và tự kết nối lại.
- Dùng được bằng bàn phím: Tab giữa các điều khiển, Escape đóng hộp thoại, focus quay về nút đã mở hộp thoại.
- Nếu bấm một nút điều khiển trên dữ liệu đã cũ (task vừa đổi trạng thái), server từ chối và màn hình tự tải lại; xem lại rồi bấm lần nữa.

7. Dashboard: năm màn hình

7.1 Tổng quan `#/`

Task chia thành 4 nhóm: **Đang chạy**, **Chờ bạn** (*Duyệt plan hoặc xử lý chỗ autopilot dừng*), **Lỗi / cần xử lý** (Vướng mắc, Cần sửa, killed, timed_out…), **Xong**. Bấm thẻ task để mở Chi tiết. Dùng *Hiện thêm 10* để xem task cũ hơn.

- Task chờ duyệt plan có nhãn *Chờ duyệt plan*.
- Task autopilot đã dừng nằm trong *Chờ bạn* với nhãn *Chờ bạn · {lý do}* và một dòng thứ ba ghi chi tiết dừng (nguyên văn như bảng mục 4.4).
- Task có autopilot đang chạy có thêm huy hiệu ◐ **Autopilot** sau ba ô Lead / Dev / QA.

7.2 Giao việc `#/assign`

1. **Yêu cầu (tiếng Việt):** mô tả việc cần làm.
2. **Dự án:** gõ tên, alias hoặc tên không dấu rồi **chọn trong danh sách gợi ý** (chuột, hoặc ↑/↓ rồi Enter). Dự án *thiếu thư mục* hiện mờ và không chọn được.
3. **Quy trình (mới, 5.2):** danh sách chọn, mỗi dòng có tên, nhãn (*mặc định của dự án*, *có sẵn*, *từ workflows.json*, *autopilot / chạy tay*), dòng giai đoạn và dòng giới hạn, ví dụ *plan ⇄ review → Bạn → code ⇄ review → QA ⇄ sửa / 3 vòng · 60 phút · $20*. Mặc định chọn quy trình của dự án, không có thì `mac-dinh`. Đổi dự án sẽ chọn lại mặc định của dự án mới, trừ khi bạn đã tự chọn. Dòng dưới cho biết *"Quy trình này tự chạy autopilot và luôn dừng ở bước bạn duyệt plan."* hoặc *"Quy trình này chạy tay: bạn bấm từng bước."*
4. **Tên task (slug):** tự tạo từ yêu cầu giống CLI; sửa được.
5. Kiểm tra khung thông tin (dự án, nhánh hiện tại, profile) rồi bấm **Giao việc**. Quy trình autopilot chạy `agent-brain auto --start`; quy trình chạy tay chạy Lead.

Tên khớp nhiều dự án thì màn hình hiện *"Tên khớp nhiều dự án — chọn một"* và nút Giao việc bị khóa. Nút bị khóa luôn kèm lý do (dự án đang bận, đang chạy autopilot, đã có run personal…). `workflows.json` có lỗi thì hiện khung hổ phách *"workflows.json có lỗi"*, hai quy trình có sẵn vẫn chọn được. Không tải được danh sách: *"Không tải được danh sách quy trình. Task sẽ chạy bằng quy trình mặc định của dự án."*

7.3 Chi tiết task `#/task/N`

Thứ tự từ trên xuống: thẻ tiêu đề → **biểu ngữ dừng** (chỉ khi autopilot dừng) → **Pipeline** → khung Vướng mắc / Cần sửa → **Sổ lượt chạy** → Kế hoạch (.plan.md) → Báo cáo QA → Log realtime.

Đầu thẻ Pipeline

- **Huy hiệu autopilot:** *Autopilot: tắt · Autopilot: đang chạy · Autopilot: đang dừng* (chờ run hiện tại xong) · *Autopilot: chờ bạn duyệt plan · Autopilot: chờ bạn · {lý do} · Autopilot: đã dừng* (driver mất, ví dụ `serve` tắt giữa chừng — bấm *Bật autopilot* để chạy tiếp từ bước dở) · *Autopilot: xong*.
- **Nút Bật autopilot / Dừng autopilot** cạnh huy hiệu. *Dừng autopilot* là dừng mềm: run hiện tại chạy nốt rồi autopilot không bắt đầu bước mới (muốn dừng ngay: Kill). Khi autopilot đang chờ ở cổng, *Dừng autopilot* **tắt** autopilot cho task: *"Đã tắt autopilot cho task này — Duyệt plan giờ chỉ ghi nhận, không tự chạy tiếp. Bấm Bật autopilot để bật lại."*
- **Dòng quy trình:** *Quy trình: mac-dinh · plan ⇄ review → Bạn → …*. Task tạo trước 5.2: *Quy trình: chưa có (task tạo trước Phase 5.2) — chạy tay từng bước.*
- **Dòng ngân sách:** *Ngân sách: M / B phút · $X / $Y*, thêm *sắp hết* (≥ 80 %) hoặc *đã hết* (≥ 100 %). Có ≈ trước số tiền khi gồm chi phí ước tính từ token của run bị dừng; khi đó số này có thể lớn hơn *Tổng đến giờ* ở Sổ lượt chạy.

Luồng pipeline vẽ theo bản sao quy trình của task: *Bắt đầu* → nhóm plan → *Bạn* → nhóm build → nhóm QA. Mỗi nhóm vòng lặp chứa hai thẻ xếp chồng và chân nhóm ⟲ *vòng N / cap* (hoặc *chưa chạy*; quy trình không có vòng lặp chỉ ghi ⟲ *vòng N*).

Thẻ (vai trò)	Dòng kết quả có thể thấy	Dòng số liệu thêm
Bắt đầu	*bạn đã giao*	giờ giao
Lên plan (lead)	*chưa bắt đầu · đang lập plan · plan đã commit · đã dừng · Lead lỗi · vi phạm giao thức*	—
Review plan (plan-review)	*đang review plan · duyệt · sửa lại · đã dừng · Review lỗi · vi phạm giao thức*	*b chặn · a góp ý*
Bạn	*chưa có plan · chờ review plan · chờ duyệt plan · đã duyệt plan*	giờ
Code (dev)	*đang code · đang sửa theo review · n lượt · vướng mắc · đã dừng · Dev lỗi · vi phạm giao thức*	—
Review code (code-review)	*đang review code · đạt · sửa lại · đã dừng · Review lỗi*	*b chặn · a góp ý*
QA (qa)	*đang kiểm tra · đạt · cần sửa · đã dừng · QA lỗi*	*s kiểm thử · f lỗi*
Sửa (dev · fix)	*không cần sửa · đang sửa · n lượt · vướng mắc · Dev lỗi*	—

- Mỗi thẻ có dòng *phút · chi phí* = tổng các run của thẻ đó. Số đếm không biết thì không hiện (không bao giờ hiện 0 thay cho "không biết").
- **Đúng một thẻ có viền chàm**: bước đang chạy, bước sắp chạy, thẻ *Bạn* khi chờ duyệt, hoặc bước đã dừng. Nhóm nơi autopilot dừng có **viền hổ phách** và dòng lý do ở chân nhóm.
- **Chú thích (7 mục):** ✓ xong · ◐ đang chạy · ● chờ bạn · ■ đã dừng / lỗi · ○ chưa bắt đầu · ○ chưa tự động · ! vướng mắc / cần sửa / sửa lại. Hàng *Chưa tự động* chỉ còn *PR nháp · chưa tự động*.
- Trên màn hình rộng luồng nằm một hàng; hẹp hơn 1024 px thì cuộn ngang trong khung *Pipeline của task*; dưới 640 px các nhóm xếp dọc.

Thẻ Bạn (cổng duyệt plan) — khi *chờ duyệt plan* có hai nút:

- **Duyệt plan** (không có hộp xác nhận): ghi duyệt plan hiện tại. Autopilot còn bật → autopilot chạy tiếp Code, Review code, QA và Sửa (cần đúng nhánh task; nếu autopilot không bật được thì bản duyệt được hoàn lại). Autopilot đã tắt → chỉ ghi nhận.
- **Trả về Lead**: mở hộp *"Trả plan về Lead cho task {slug}?"* với ô *Ghi chú cho Lead (không bắt buộc)*, **tối đa 500 ký tự**. Nút *Trả về Lead* / *Giữ plan hiện tại*. Autopilot còn bật → chạy lại vòng plan ⇄ review plan; đã tắt → chạy Lead tay.
- Dòng chú thích ngay dưới luồng giải thích cổng, ví dụ *"Autopilot dừng ở đây để bạn duyệt plan. Duyệt plan rồi autopilot tự chạy Code, Review code, QA và Sửa, tối đa 3 vòng mỗi bước, trong 60 phút hoặc $20."* Nếu vòng plan dừng trước khi review duyệt: *"Plan chưa qua review. Bạn có thể duyệt plan hiện tại hoặc trả về Lead."*
- Trong lúc autopilot chờ ở cổng, dải nút chạy tay ẩn *Chạy Dev* và *Chạy lại Lead*.

Biểu ngữ dừng (hổ phách): tiêu đề *Autopilot dừng · {lý do}*, chi tiết nguyên văn (mục 4.4), dòng ngân sách nếu lý do là *hết ngân sách*, và ba nút: **Chạy thêm 1 vòng** (với hết ngân sách có ghi *"Chạy vượt ngân sách một lần, chỉ cho vòng này."*), **Chạy lại Lead** (có hộp xác nhận), **Dừng hẳn** (không hộp xác nhận; bỏ biểu ngữ, autopilot tắt — *Bật autopilot* để chạy tiếp).

Dải hành động (*Bước hiện tại: …*): các nút chạy tay *Chạy Dev*, *Chạy QA*, *Chạy lại Lead* và **Kill**. Mỗi nút bị khóa đều kèm lý do. Hộp Kill mặc định chọn nút an toàn (*Giữ chạy tiếp*); khi autopilot chạy, hộp ghi thêm *Autopilot*

cũng dừng với lý do "dừng bởi bạn".

Sổ lượt chạy (mới từ 5.1, mở rộng ở 5.2)

- Nhóm theo chu kỳ: *Chu kỳ 1 · plan đầu tiên*, *Chu kỳ k · plan lại*, *Chu kỳ k · build*, *Chu kỳ k · QA*; nhóm plan ghi thêm *bạn đã duyệt* hoặc *chờ bạn* bên phải. Chu kỳ tăng mỗi lần Lead chạy lại sau build hoặc QA.
- Mỗi dòng: *Vòng n · {plan | review plan | code | sửa theo review | review code | QA | sửa theo QA}: {kết luận}*; bên phải số đếm và số liệu, ví dụ *1 chặn · 4 góp ý · 41 lượt · 2 phút · $1.32*. Kết luận: *đang chạy, duyệt, đạt, sửa lại, cần sửa, vướng mắc, vi phạm giao thức, đã dừng, lỗi, xong*.
- Bấm một dòng để xem log của run đó (dòng ghi *Đang xem*) và hiện dòng token *Token: vào X · ra Y · cache R / W*.
- Dòng cuối *Tổng đến giờ — N lượt chạy claude · M phút · $X*. Run bị Kill hiện chi phí — và không cộng vào tổng; khi đó có dòng *"Chưa tính chi phí của k run bị dừng hoặc chưa có kết quả."* Hiện 20 run mới nhất; *Hiện thêm 20* để xem cũ hơn.

Kế hoạch, báo cáo QA, log: plan và báo cáo QA hiển thị chỉ đọc (sửa bằng editor rồi commit). Log tự cuộn theo dòng mới; cuộn lên để tạm dừng, *Về cuối (live)* để theo lại; bấm một dòng log để xem JSON thô.

7.4 Thống kê `#/stats`

Biểu đồ token và chi phí tương đương theo ngày, theo vai trò (gồm cả Review plan, Review code), theo dự án và theo profile. Cùng số liệu với `agent-brain stats`; theo bộ lọc profile ở thanh trên.

7.5 Dự án `#/projects`

- Danh sách dự án trong `projects_map.json`, dự án mất thư mục được đánh dấu. Mỗi dự án có dòng *Quy trình mặc định: {tên}*; chú thích *"Profile và quy trình mặc định chỉ đổi trong projects_map.json"*.
- **Sửa alias** rồi bấm *Lưu alias*. Alias trùng với dự án khác bị từ chối. Nếu danh mục vừa bị sửa ở nơi khác, thay đổi của bạn không ghi đè: dashboard báo lỗi, tải lại dữ liệu mới, và bạn nhập lại alias.
- **Quét lại** = `agent-brain projects scan`.
- **Thẻ Quy trình (mới, chỉ xem):** mỗi quy trình có tên, nhãn (*có sẵn* / *từ workflows.json* / *ghi đè quy trình có sẵn*, *autopilot* / *chạy tay*), dòng giai đoạn, giới hạn và *Mặc định của: {dự án}*. Sửa `workflows.json` trong editor rồi bấm **Đọc lại**. Lỗi của file hiện ở khung *workflows.json có lỗi*.
- Không sửa được profile, quy trình mặc định, id, đường dẫn hay loại dự án trên web (sửa trong file).

8. Dữ liệu, file và an toàn

Vị trí file

Đường dẫn	Nội dung
`~/.config/agent-brain/projects_map.json`	Danh mục dự án (kèm quy trình mặc định).
`~/.config/agent-brain/config.json`	Cấu hình (tùy chọn).
`~/.config/agent-brain/workflows.json`	Quy trình của bạn (tùy chọn).
`~/.local/share/agent-brain/brain.db`	SQLite: task (kèm bản sao quy trình, duyệt plan, trạng thái autopilot), run, sự kiện, token. Quyền `0600`.
`~/.local/share/agent-brain/runs/`	Log chi tiết từng run. Quyền thư mục `0700`.
`serve.lock` (thư mục dữ liệu)	Khóa một-dashboard: PID, cổng, giờ khởi động. Không chứa token.
`.plan.md`, `.plan-review.md`, `.code-review.md`, `.qa-report.md` (gốc repo)	Plan của Lead, báo cáo của hai vai trò review và của QA, trên nhánh task.

Mọi dòng stream-json thô từ Claude được lưu nguyên vẹn (cột `raw_json` của bảng `events`) để kiểm tra lại bất cứ lúc nào.

Phạm vi ghi của từng vai trò

Vai trò	Được ghi
Lead	Chỉ `.plan.md`.
Review plan / Review code	Đọc toàn repo; **chỉ ghi báo cáo của mình** (`.plan-review.md` / `.code-review.md`).
Dev (code)	Các file trong bảng *Thay đổi* của plan.
Dev chế độ sửa (`--fix`)	File của plan **cộng** file được báo cáo nêu tên: danh sách `files` ở front matter của báo cáo (có thể là file mới), và đường dẫn trong dấu backtick ở thân báo cáo — nhưng đường dẫn ở thân **chỉ được tính khi file đó đã thay đổi** từ commit plan. Tối đa 20 file.
QA	Chỉ `.qa-report.md`; không sửa code.

Chế độ sửa không bao giờ được ghi (không phân biệt hoa thường): file bí mật (`.env*`, `id_rsa*`, `id_ed25519*`, `*.pem`, `*.key`); file CI / hook / editor / build (`.github/`, `.gitlab/`, `.gitlab-ci.yml`, `.circleci/`, `.husky/`, `.githooks/`, `.gitmodules`, `.gitattributes`, `Makefile`, `package.json`, `.npmrc`, `.pre-commit-config.yaml`, `.vscode/`, `.devcontainer/`, `.claude/`); symlink hỏng hoặc trỏ tới file nhạy cảm; file đã có mà git không theo dõi. Vì vậy một báo cáo review không thể mở rộng quyền ghi sang CI, hook hay script build.

Kiểm tra lệnh Bash (mọi vai trò)

Hook `guard` kiểm tra mọi lệnh Bash agent muốn chạy:

- **Một lệnh đơn trên một dòng:** không xuống dòng, không `;` `&` `&&` `|` `(` `)` `{` `}`, không chuyển hướng `<` `>`, không `$` (kể cả `$(…)`) hay dấu backtick, không gán biến môi trường ở đầu lệnh.
- **Git chỉ được:** `git diff`, `log`, `show`, `status`, `add`, `commit` (trước tên lệnh chỉ cho `--no-pager` / `-P`). Không ký tự đại diện `*` `?` `[` ngoài dấu nháy. Không gọi git qua lệnh bọc (`env`, `timeout`, `sh -c`,

`xargs`, `find`, `sudo`, ...) hay `git-<lệnh>`.

- **Tùy chọn bị cấm:** `diff` / `log` / `show` với `--output`, `--ext-diff`, `--textconv`, `--no-index`; `commit` với `--amend`, `-F` / `--file`, `-t` / `--template`, `--pathspec-from-file`, các dạng mở trình soạn thảo; `add` với `-f`, `-p`, `-i`, `-e`, `--pathspec-from-file`.
- **Commit phải có `-m` một dòng** (hoặc `-C` / `--reuse-message`).
- Lệnh git bạn thêm vào `common_args` (ví dụ `git push`) vẫn bị hook từ chối.

Nguyên tắc an toàn

- Không quy trình nào tắt được: Kill chỉ qua cơ chế `kill` của agent-brain; một run mỗi dự án; một run `personal`; phạm vi ghi của từng vai trò; agent không bao giờ chạy với `--dangerously-skip-permissions`; autopilot luôn dừng ở cổng duyệt plan.
- Token dashboard tương đương quyền chạy lệnh trên máy bạn — giữ kín (xem mục 6).
- **Rủi ro đã chấp nhận:** cookie không tách theo cổng (RFC 6265). Một dịch vụ web khác đang chạy trên `127.0.0.1` về lý thuyết có thể nhận được cookie dashboard. Khi không dùng dashboard, hãy tắt `serve`; khi đang chạy, tránh mở trang của dịch vụ cục bộ lạ trên cùng trình duyệt.
- Agent không được gán API key: các biến `ANTHROPIC_*` bị bỏ đặt, agent dùng tài khoản đăng nhập của profile.
- Không dừng tiến trình agent hay autopilot bằng `pkill` / `killall`; dùng `agent-brain kill` hoặc nút Kill để dọn sạch cây tiến trình và cập nhật database.

9. Xử lý sự cố

Hiện tượng	Cách xử lý
Dashboard báo **401** / phiên hết hạn	Mở lại URL đầy đủ có `?token=` đang in ở terminal `serve`. Sau khi khởi động lại serve, token cũ không dùng được nữa.
421 hoặc không đăng nhập được qua `localhost`	Dùng đúng `http://127.0.0.1:PORT/…`.
`serve` báo đã có dashboard khác chạy	Dùng dashboard đó (PID và cổng được in ra), hoặc dừng nó bằng Ctrl+C ở terminal của nó.
Cổng bị chiếm (*Address in use*)	`--port <cổng>`, `--port 0`, hoặc khóa `dashboard_port`.
Máy khác trong LAN không vào được	Đúng thiết kế: dashboard chỉ nghe `127.0.0.1`.
Mã thoát **2** khi chạy lệnh	Tên dự án mơ hồ — dùng id trong danh sách được in ra.
Mã thoát **1**, "Không tìm thấy .plan.md"	Chạy Lead trước, hoặc chuyển sang đúng nhánh `task/<slug>`.
Dự án bị từ chối vì profile rỗng	Đặt `"profile": "work"` hoặc `"personal"` trong `projects_map.json`.
Không có quy trình 'x' — các quy trình có: …	Sửa tên trong `--workflow`, hoặc thêm quy trình vào `workflows.json`. Không có nhánh hay task nào được tạo.
Khung *workflows.json có lỗi* / dòng *Cảnh báo workflows.json: …*	Sửa đúng dòng / khóa được nêu (mục 3.4), rồi bấm *Đọc lại* ở Dự án. Trong lúc đó `mac-dinh` và `nhanh` vẫn dùng được.
Không ở nhánh task 'task/…' (nhánh hiện tại: '…') — không đoán	`git checkout task/<slug>` trong repo dự án rồi duyệt / chạy lại. Chỉ muốn ghi duyệt: `agent-brain approve N --no-continue`.
Dự án … đang chạy autopilot cho task N — …	Chờ autopilot dừng, hoặc bấm *Dừng autopilot* trên task N.
Autopilot dừng *run lỗi* với *Working tree có thay đổi chưa commit*	Thường là báo cáo QA không có kết luận được giữ lại. Xem `git status`, đọc rồi xóa hoặc commit file đó, sau đó *Chạy thêm 1 vòng*.
Huy hiệu *Autopilot: đã dừng*, không có biểu ngữ	`serve` đã tắt khi autopilot chạy. Bấm *Bật autopilot* hoặc `agent-brain auto N`.
Duyệt plan xong mà không chạy tiếp	Autopilot đang tắt cho task (đúng thiết kế). Bấm *Bật autopilot* hoặc `agent-brain auto N`.
Không duyệt được: *Task tạo trước Phase 5.2 — …*	Task cũ không có cổng duyệt: bấm *Chạy Dev* để duyệt như trước.
Nút bấm báo dữ liệu cũ, màn hình tự tải lại	Task vừa đổi trạng thái. Xem trạng thái mới rồi bấm lại nếu vẫn cần.
Nút Chạy Dev/QA bị khóa hoặc không thấy	Đọc lý do cạnh nút: dự án đang bận, đã có run personal, plan chưa commit, autopilot đang chờ ở cổng…
Run treo lâu	Bấm Kill (hoặc `agent-brain kill TASK_ID`). Run tự dừng khi quá `timeout_seconds`.
Dữ liệu dashboard không cập nhật	Thấy thông báo "đang thử lại" thì chờ; vẫn không được thì tải lại trang (F5).

10. Hạn chế đã biết (trước khi báo lỗi)

Các mục dưới đây đã được ghi nhận ở các lần review Phase 5, 5.1 và 5.2 (file `*-REVIEW.md`, `*-REVIEW-DISPOSITION.md` trong `.planning/phases/`) và lần nghiệm thu 5.2 (`protocol-log/nghiem-thu-p5-2.md`), và đang để ngỏ hoặc hoãn. Nếu bạn gặp đúng hiện tượng này, chỉ cần ghi thêm bằng chứng / ảnh hưởng thực tế, không cần báo mới.

Autopilot, quy trình, guard (Phase 5.2)

Mã	Hiện tượng bạn có thể thấy	Ghi chú
QA-kết luận	Báo cáo QA không có kết luận được **giữ lại** làm bằng chứng, nên working tree bẩn và bước Lead / Dev sau bị chặn (autopilot dừng *run lỗi*).	Đọc, rồi xóa hoặc commit `.qa-report.md`; sau đó chạy tiếp.
Guard	Agent thử `git ls-files` hoặc chuỗi `ls … ; cat …` và bị guard từ chối.	Vô hại, chỉ tốn một lượt của agent; run vẫn tiếp tục. `git ls-files` (chỉ đọc) có thể được cho phép sau.
Ngân sách	Run bị Kill mà chưa ghi được token nào được tính **$0** vào ngân sách autopilot.	Ngân sách có thể thấp hơn chi phí thật một chút trong trường hợp này.
IN-08	Kill trên dashboard và driver autopilot có thể cùng gọi lệnh dừng trên một run.	Hoãn; review đánh giá vô hại.
IN-04	Mã nguồn bộ quyết định autopilot (`decide.rs`) rất dài, có vài trường không dùng.	Hoãn; chỉ là dọn mã, không ảnh hưởng hành vi.

Pipeline và Sổ lượt chạy (Phase 5.1)

Mã	Hiện tượng bạn có thể thấy	Ghi chú
5.1 IN-03	Chi phí trên thẻ pipeline chỉ cộng các run biết chi phí; run bị Kill không được cộng và thẻ không báo điều đó.	Sổ lượt chạy có dòng *Chưa tính chi phí của k run…*; dòng ngân sách có ≈.
5.1 IN-06	Khi mở log của một run vừa bắt đầu, dòng trạng thái log có thể còn ghi vai trò / trạng thái của run trước trong vài giây.	Tự đúng lại ở lần cập nhật sau.

Dashboard (Phase 5)

Mã	Hiện tượng bạn có thể thấy	Ghi chú
WR-04	Giao việc: gõ tên rồi nhấn Enter quá nhanh (< 0,2 giây) có thể chọn từ danh sách gợi ý cũ.	Luôn kiểm tra dự án trong khung thông tin trước khi bấm Giao việc.
WR-05	Hai lần giao việc cho hai dự án `personal` gần như cùng lúc có thể cùng được nhận.	Lớp chạy `run` kiểm tra lại; vẫn nên tránh.
WR-06	Hai dự án có nhánh task trùng tên có thể làm sai trạng thái "đang chạy".	Hiếm.
WR-11	Kill đúng lúc run đổi bước có thể dừng run mới của cùng task.	Cửa sổ vài mili giây.
WR-09/10	Chuyển màn hình hoặc mở JSON thô liên tục quá nhanh: phản hồi cũ có thể ghi đè màn hình mới.	Tải lại trang là hết. (Màn hình task đã có chặn phản hồi cũ.)

WR-01	Giá trị `--since` cực lớn có thể cho kết quả sai.	Dùng giá trị hợp lý (ví dụ `90d`).
WR-13	Lỗi 500 của các API dự án, run và thống kê có thể hiện thông báo kỹ thuật kèm đường dẫn nội bộ. (API task và quy trình đã trả thông báo cố định.)	Xóa đường dẫn riêng tư trước khi dán vào issue.
UI-04	Nhãn ngày trên biểu đồ Thống kê chưa được kiểm với dữ liệu > 14 ngày.	Báo nếu thấy chữ chồng nhau.

Đã sửa từ bản trước của tài liệu: focus bàn phím quay về nút sau khi đóng hộp thoại (WR-03), danh mục HTTP API trong README (WR-02), mũi tên stepper mờ (stepper đã thay bằng pipeline). Các lỗi 5.1 về cổng duyệt và pipeline (WR-01…WR-05) đã sửa trong Phase 5.2.

Chưa có (chủ ý hoặc để phase sau): tự tạo PR nháp, QA tự viết và chạy kiểm thử mới, sửa quy trình trên dashboard, sửa plan trên web, thông báo desktop khi task cần bạn, chạy engine khác ngoài Claude (Phase 6), tối ưu token bằng tóm tắt/handover (Phase 4).

11. Review, báo lỗi và đề xuất nâng cấp

11.1 Gợi ý review theo từng phần

Phần	Những điều nên thử
Giao việc & định tuyến	Tên có dấu/không dấu, alias, tên khớp nhiều dự án, dự án mất thư mục, chọn quy trình, quy trình mặc định của dự án.
Chất lượng plan & Review plan	Plan có đủ bảng Thay đổi? Review plan có bắt được lỗi thật? Có lần *revise* vô lý hay *approve* sai? Lead có xử lý đủ mục chặn ở vòng 2?
Review code & chế độ sửa	Review code có hữu ích? Dev `--fix` có sửa đúng phạm vi? Có file bị *Bỏ qua (không được phép)* mà lẽ ra nên cho?
QA	Báo cáo có bắt được lỗi thật? Có lần "Đạt" sai? Số *kiểm thử / lỗi* có đúng?
Autopilot	Có dừng đúng ở cổng duyệt? Lý do dừng và chi tiết có dễ hiểu? *Chạy thêm 1 vòng*, *Dừng hẳn*, *Bật autopilot* làm đúng? Ngân sách có khớp Sổ lượt chạy?
Kill & độ bền	Kill giữa chừng (cả khi autopilot chạy), Ctrl+C ở terminal `auto`, đóng terminal, tắt dashboard khi đang chạy; sau đó kiểm tra không còn tiến trình `claude` sót.
Dashboard	Pipeline, Sổ lượt chạy, log trực tiếp, tải lại trang giữa chừng, giao diện sáng/tối, màn hình hẹp (480–768 px), dùng toàn bàn phím.
Token & chi phí	Vai trò nào tốn nhất? Review có đáng tiền? Giới hạn 3 vòng / 60 phút / $20 có hợp lý với việc của bạn? (Dữ liệu này là đầu vào cho Phase 4.)

11.2 Thu thập thông tin trước khi báo

```
agent-brain --version
agent-brain status PROJECT               # trạng thái repo, run gần nhất
agent-brain stats --since 24h --json     # nếu liên quan token/chi phí
claude --version                         # phiên bản Claude CLI
```

- **Số task và số run** (in ở terminal, hoặc trên URL `#/task/N` và Sổ lượt chạy của dashboard).
- **Quy trình** của task (dòng *Quy trình: …*) và, nếu có, dòng *Autopilot dừng · {lý do}: {chi tiết}* nguyên văn.
- Mã thoát của lệnh: `echo $?` ngay sau khi lệnh kết thúc.
- Log của run trong `~/.local/share/agent-brain/runs/` và phần stderr của terminal `serve` nếu lỗi ở dashboard.
- Báo cáo liên quan (`.plan-review.md`, `.code-review.md`, `.qa-report.md`) nếu lỗi nằm ở kết luận.
- Ảnh chụp màn hình (đã che token) và kích thước cửa sổ trình duyệt nếu là lỗi giao diện.

> **Trước khi gửi: xóa thông tin nhạy cảm**
> Không đưa token dashboard (`?token=…`), cookie, API key, nội dung mã nguồn riêng của khách hàng hay đường dẫn home đầy đủ vào issue.

11.3 Mẫu báo lỗi

```
Tiêu đề: [dashboard|cli|plan|review|qa|autopilot|kill|stats] Mô tả ngắn

Phiên bản: agent-brain 0.1.0, claude x.y.z, trình duyệt …
Profile: work | personal      Task/Run: task 12, run 31
Quy trình: mac-dinh | nhanh | …    Autopilot: bật | tắt | dừng · {lý do}
Mức độ: chặn | nặng | nhẹ | thẩm mỹ

Các bước tái hiện:
1. …
2. …

Kết quả mong đợi: …
Kết quả thực tế: …  (mã thoát, thông báo lỗi, dòng "Autopilot dừng · …")
Bằng chứng: log/ảnh (đã che token)
Liên quan hạn chế đã biết? (WR-xx / IN-xx / không)
```

11.4 Mẫu đề xuất nâng cấp

```
Tiêu đề: [đề xuất] …
Vấn đề hiện tại (đau ở đâu, bao lâu gặp một lần): …
Đề xuất: …
Lợi ích đo được (thời gian, token, chi phí, số lần thao tác): …
Phạm vi: CLI | dashboard | giao thức | quy trình | autopilot | cấu hình
Ưu tiên của bạn: cao | vừa | thấp
```

11.5 Mức độ

Mức	Khi nào
Chặn	Không chạy được Lead/Dev/QA, chạy nhầm dự án hoặc nhầm nhánh, autopilot vượt cổng duyệt, mất dữ liệu, tiến trình sót sau Kill.
Nặng	Sai trạng thái, sai số liệu hay ngân sách, chức năng không làm đúng nhưng có cách vòng.
Nhẹ	Chậm, khó dùng, thông báo khó hiểu.
Thẩm mỹ	Màu, chữ, khoảng cách, căn chỉnh.

11.6 Gửi ở đâu

Tạo issue trên kho mã **github.com/vinhhx/quang-vy-ai-orchestration** (tab Issues), hoặc gửi trực tiếp cho người phụ trách dự án. Mỗi lỗi một issue giúp lập kế hoạch sửa (fix plan) dễ hơn.

12. Phụ lục

A. Mã thoát

Mã	Tên	Ý nghĩa
0	EXIT_OK	Thành công (kể cả QA Đạt / Cần sửa và review approve / revise hợp lệ). Với `auto` / `approve`: autopilot dừng ở cổng duyệt plan, chạy xong (QA đạt), hoặc chỉ ghi nhận duyệt.
1	EXIT_ERROR	Lỗi thực thi, cấu hình sai, thiếu file, điều kiện git không thỏa, quy trình không tồn tại, bị từ chối vì dự án / profile đang bận, hoặc `approve` bị từ chối (ví dụ sai nhánh). Khi autopilot đang chạy mà gặp sai nhánh thì là mã 4 (*run lỗi*).
2	EXIT_ROUTING	Tên dự án khớp 0 hoặc nhiều hơn 1 dự án (Fallback Ask).
3	EXIT_PROTOCOL	Vi phạm giao thức: sửa file ngoài phạm vi được phép, hoặc báo cáo sai mẫu.
4	EXIT_BLOCKED	`run dev`: Vướng mắc, đã ghi vào `.plan.md`. `auto` / `auto --start` / `approve`: **autopilot đã dừng và chờ bạn** (hết vòng, hết ngân sách, vi phạm giao thức, vướng mắc, dừng bởi bạn, run lỗi).
130	EXIT_INTERRUPTED	Dừng bởi Ctrl+C hoặc `agent-brain kill`. Với `auto`: driver bị ngắt (SIGINT / SIGTERM / SIGHUP), chạy tiếp bằng `agent-brain auto N`.

B. Khóa cấu hình đầy đủ (`config.json`)

Khóa	Mặc định	Ý nghĩa
`claude_bin`	`"claude"`	Lệnh Claude CLI.
`models`	lead opus, dev sonnet, qa sonnet; `plan-review` / `code-review` opus khi không ghi	Model theo vai trò.
`inspect_model`	`"sonnet"`	Model cho `inspect`.
`profiles`	work, personal	`claude_config_dir` và các biến môi trường bị bỏ đặt cho từng profile.
`common_args`	`["--strict-mcp-config"]`	Đối số chung cho Claude CLI.
`max_turns`	null cho lead, dev, qa, plan-review, code-review	Giới hạn lượt theo vai trò (null = không giới hạn).
`timeout_seconds`	lead 1800, dev 1800, qa 1200, inspect 300, plan-review 1200, code-review 1500	Trần thời gian của một run (giây). Vai trò không có khóa dùng 1800.
`autopilot.max_rounds`	3	Số vòng tối đa mỗi vòng lặp (1–10).
`autopilot.budget_minutes`	60	Tổng phút của task (1–1440).
`autopilot.budget_usd`	20	Tổng chi phí tương đương của task (> 0, ≤ 1000).
`pricing`	opus 4 / 20 / 0.2 / 8; sonnet 2 / 10 / 0.1 / 4; haiku 0.1 / 0.5 / 0.005 / 0.2	USD mỗi triệu token: `input` / `output` / `cache_read` / `cache_write`. Dùng ước tính chi phí run không có kết quả cuối.

`runs_dir`	null → `~/.local/share/agent-brain/runs/`	Thư mục log run.
`kill_grace_secs`	3	Chờ giữa SIGINT và SIGKILL.
`inspect_max_diff_bytes`	100000	Giới hạn diff gửi cho `inspect`.
`dashboard_port`	7437	Ưu tiên: `--port` > khóa này > 7437.

Giá trị autopilot ngoài khoảng làm lệnh báo lỗi cấu hình. Quy trình trong `workflows.json` có thể ghi đè `max_rounds`, `budget_minutes`, `budget_usd` cho riêng mình (mục 3.4).

C. HTTP API của dashboard

Mọi route cần cookie phiên (trừ `GET /?token=…` dùng để lấy cookie). Lệnh ghi (POST/PUT) cần thân JSON và còn kiểm tra Origin và Content-Type; thân của route task có khóa lạ bị từ chối. Các nút điều khiển gửi `rev` của task đang hiển thị: `rev` cũ trả `409` (`"kind": "stale"`). Dành cho người phát triển; người dùng thường không cần gọi trực tiếp.

Route	Việc làm
`GET /api/tasks`	Danh sách task (nhóm, nhãn, trạng thái autopilot, lý do dừng) và trạng thái bận của dự án (lọc `?profile=`, `&limit=`).
`POST /api/tasks`	Giao việc. Body: `project`, `request`, `name`, `workflow` (hai khóa sau không bắt buộc). Quy trình autopilot chạy `auto --start`, còn lại chạy Lead. Quy trình không tồn tại → `422`.
`GET /api/tasks/{id}`	Chi tiết task: các bước, plan, báo cáo QA, hành động và nút điều khiển kèm lý do khóa, trạng thái autopilot, `rev`.
`GET /api/tasks/{id}/pipeline`	Luồng pipeline theo quy trình: nhóm và thẻ, vòng / giới hạn, cổng duyệt, thẻ hiện tại, ngân sách, số lượt chạy.
`POST /api/tasks/{id}/dev|qa|relead`	Chạy Dev, QA, hoặc chạy lại Lead (autopilot còn bật thì `auto N --replan`).
`POST /api/tasks/{id}/approve`	Duyệt plan (`rev`); autopilot còn bật thì chạy tiếp.
`POST /api/tasks/{id}/return`	Trả về Lead (`note` ≤ 500 ký tự, `rev`).
`POST /api/tasks/{id}/auto`	Bật autopilot.
`POST /api/tasks/{id}/auto/stop`	Dừng autopilot (dừng mềm); ở cổng duyệt thì tắt autopilot cho task.
`POST /api/tasks/{id}/auto/more-round`	Chạy thêm 1 vòng sau khi autopilot dừng.
`POST /api/tasks/{id}/auto/end`	Dừng hẳn (bỏ biểu ngữ, tắt autopilot).
`GET /api/preflight`	Kiểm tra trước khi giao việc (dự án, nhánh, profile, lý do khóa).
`GET /api/slug?text=…`	Tạo slug từ văn bản tiếng Việt.
`GET /api/jobs/{id}`	Trạng thái job do dashboard khởi chạy.
`GET /api/runs/{id}`	Thông tin run.
`GET /api/runs/{id}/stream`	Log trực tiếp (Server-Sent Events).
`GET /api/runs/{id}/events/{seq}/raw`	JSON thô của một sự kiện.
`POST /api/runs/{id}/kill`	Dừng run (cùng cơ chế với `agent-brain kill`); autopilot của task dừng với *dừng bởi bạn*.

`GET /api/projects`	Danh mục dự án (kèm quy trình mặc định) và `etag`.
`PUT /api/projects/{id}`	Sửa alias. Body: `aliases`, `etag`.
`POST /api/projects/scan`	Quét lại thư mục dự án.
`GET /api/projects/suggest?q=…`	Gợi ý dự án theo độ khớp tên/alias.
`GET /api/projects/{id}/status`	Trạng thái git nhanh (giống `status`).
`GET /api/stats?profile=…&since=…`	Thống kê token và chi phí.
`GET /api/workflows`	Danh sách quy trình (có sẵn trước, rồi theo thứ tự trong file), lỗi của `workflows.json`, dự án nào dùng quy trình nào làm mặc định. Chỉ đọc.